NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 2.
CSS fundamentals: box model, cascade, specificity, and AI-assisted layout

Modern Web Development: HTML, CSS, JS/TS

1. Aim

Explore the fundamental mechanisms of CSS (box model, cascade, specificity, inheritance) and learn to apply them deliberately when building web pages. Master an AI-assisted development approach while keeping control of the architecture and quality of your code.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- Understand and explain how element sizes are determined (box model).
- Know how style conflicts are resolved (cascade and specificity).
- Understand how property inheritance works.
- Use CSS variables to create a consistent design.
- Use browser developer tools (DevTools) to debug CSS.
- Use AI assistants effectively to generate markup and styles, while critically evaluating and refactoring the generated code.

3. Lab tasks

1. **Run a series of experiments** on core CSS concepts, using web.dev materials and experiments in the browser.
2. **Build the layout with AI assistance**, applying the principles you explored. The layout and its requirements are described in a separate document — the assignment "Pricing section".
3. **Write a research report** that shows your understanding of the material based on your own observations and DevTools.
4. **Defend your work orally**, explaining how CSS works in your own project.

4. Research tasks

Before starting the practical part, study the theory and run the experiments. Add the results (observations and conclusions) to your report.

4.1. Box model

- Study CSS Box Model — web.dev.
- Create two `<div>` blocks with the same width and height. Set `box-sizing: content-box` on one and `border-box` on the other. Give both `padding` and `border`.
- Open DevTools (the Computed tab). Explain in your report why the actual rendered size of the blocks differs, and conclude which approach is more predictable for layout.

4.2. Cascade

- Study The Cascade — web.dev.
- Set the text color of one element in three ways with equal specificity (for example, through classes), but include them in a different order: through an external file, through a `<style>` tag, and by changing the order of the class declarations in the CSS itself.
- Which rule wins, and why? What happens if you change the order in which the styles are included?

4.3. Specificity

- Study Specificity — web.dev.
- Write rules for one element using a tag selector, a class selector, and an ID selector. Try to override the ID's style with a chain of several classes (for example, `.class1.class2.class3`…).
- Use the specificity calculation `(0,0,0)` to explain why an ID always beats any number of classes. Demonstrate it with a DevTools screenshot.

4.4. Inheritance

- Study Inheritance — web.dev.
- Set `font-family` and `color` on the `body` tag. Inside it, place text in `<p>`, `<a>` (link), and `<button>` tags.
- Which styles are inherited and which are not? Why does the browser not apply inheritance to certain tags, and how can you fix it (for example, with the `inherit` value)?

4.5. CSS variables

- Study the materials on CSS custom properties on web.dev: in the Learn CSS course they are collected in the Functions module, in the section "Custom properties and `var()`". For a more detailed description of variable scope and inheritance, see MDN: Using CSS custom properties.
- Declare several variables (for example, colors) in the `:root` pseudo-class. Use them in your styles. Then try to override these variables inside a specific class (for example, `.dark-card`).
- How does local scope work for CSS variables? How can you use it to create themes or component states?

4.6. Style reuse and refactoring

- **Research:** Based on the previous topics, analyze the concept of creating reusable utility classes (for example, a base `.btn` class with `.btn-primary` and `.btn-outline` modifiers).
- **Research question:** Why is a component-based approach to CSS better than writing unique styles for every individual element? How does it affect the size of the CSS file and its maintenance?

5. Practical part

Task. Build a web page based on the provided layout. You may use AI tools (ChatGPT, Claude, GitHub Copilot, and others) to generate the basic HTML structure and CSS styles. However, you are fully responsible for the quality and architecture of the code.

> **Important!** The practical implementation of the layout is done during the lab session. If you run out of time, you finish the work on your own.

Requirements for the final implementation

In the final project, you must deliberately apply the principles you explored. During the defense, you must demonstrate the following in the code and in DevTools:

1. **A correct box model.** Global use of `box-sizing: border-box` and an understanding of its effect.
2. **Working with inheritance.** Avoiding unnecessary duplication of properties (fonts, colors) that can be inherited.
3. **Deliberate use of specificity.** No `!important` declarations (except in exceptional utility cases) and no overly long selector chains.
4. **CSS variables.** Moving colors, spacing, and fonts into `:root` so the design is easy to scale.

5. **Class reuse.** Extracting shared components (for example, buttons, cards, and containers) into separate classes.
6. **No unnecessary duplication.** The code must be DRY (Don't Repeat Yourself). AI-generated code often contains duplication, so you need to refactor it.

The full description of the layout, the HTML structure, the set of CSS variables, the breakpoint, and the acceptance criteria are in a separate document: Assignment — pricing section.

6. Report requirements

Email subject:

```
[Web] Lab 2: CSS Fundamentals
```

In the email, include links to:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

> **Important!** The report for this lab is the completed and deployed layout from section 5. To defend your work, you need to understand and be able to explain:
>
> - why an element has exactly that size;
> - which rule won and why;
> - why a property was inherited;
> - why a particular CSS variable was created;
> - why a particular style was moved into a shared class.

Always include screenshots of the **DevTools** panels (the Elements, Styles, and Computed tabs) that support your conclusions.

7. Oral defense

Handing in this lab includes an oral interview about your practical implementation. Even if part of the code was generated by AI, you must be able to explain every line.

Sample defense questions (with screen sharing):

- Open DevTools. Why does this element have exactly this physical size (explain using the box model)?
- Find a crossed-out rule in DevTools. Which rule won, and why that one?

- Why did this text element get this font even though you did not set it? Show the inheritance chain.
- Why did you use a CSS variable in this particular place? What happens if you change its value in `:root`?
- Show an example where you moved styles into a shared class. What would the code look like without it?

8. Grading

1. A successful defense — **2 points**.
2. An additional **1 point** if either of the following applies:
    - you submit by **September 23, 2026**, OR
    - you do the work in class.